Wallet Ledger

Design Document

Author: Daiana Tei
HSE, Databases '23

Overview

The project provides a relational PostgreSQL database architecture for a multi-currency digital wallet. It allows users to create wallets, enforce monetary limits, and categorize cash flows, ensuring data consistency.

Core Entities

The relational schema consists of:

- **User (`person`):**

  Secured authentication data (email and password).

- **Currency (`currency`):**

  Standardized 3-letter currency codes (e.g., USD, EUR).

- **Wallet (`wallet`):**

  Financial containers holding balances, linked to a specific user and currency, enforcing maximum holding limits.

- **Category (`category`):**

  Transaction classifications (Income/Expense), supporting both system-wide defaults and user-specific custom categories.

- **Operation (`operation`):**

  The immutable ledger of financial movements, recording timestamps, amounts, and foreign keys to respective wallets and categories.

Functional Requirements

User Management

- **Registration:**

  Insertion of unique users into the database, protected by `UNIQUE` constraints on the email column.

- **Deletion:**

  Support for account deletion, utilizing cascading logic or manual procedural deletion of associated financial records to maintain referential integrity.

Currency Management

- **Currency provisioning:**

  Administrative capability to seed the database with ISO 4217 currency codes. Application-level creation is restricted to prevent ledger fragmentation.

- **Obtaining list of currencies**

Wallet Management

- **Instantiation:**

  Users can create isolated wallets. Each wallet enforces a specific currency, a maximum capacity limit (`max_limit`), and a strictly typed current `balance`.

- **Retrieval:**

  Querying specific user wallets.

- **Deletion:**

  Dropping user wallet alongside operation history.

3.4 Category Management

- **System Defaults:**

  Administrative seeding of default `INCOME` and `EXPENSE` categories (where `person_id` is `NULL`), available globally to all tenants.

- **Custom Categories:**

  Users can define custom tracking categories uniquely scoped to their `person_id`.

- **Retrieval:**

  Querying categories available to specific user.

- **Search**

- **Filtering:**

  Fetching available categories via pattern matching (`LIKE`) on substrings or by transaction type.

3.5 Ledger Operations

- **Creation:**

  Adding an operation (deposit/withdrawal) dynamically recalculates the wallet balance.

- **Deletion**

- **Historical Analysis:**

  Querying the total aggregated income and expenses over the entire account lifetime.

- **Time-Series Analysis:**

  Querying aggregated financial flow strictly for the trailing 30-day window.

Technical Architecture

Schema Definition (DDL)

Note: Referential integrity is enforced via Foreign Keys. Monetary values are strictly typed as `decimal(10,2)` to prevent floating-point precision loss.

```
CREATE TABLE person (
    id       serial       PRIMARY KEY,
    email    varchar(255) NOT NULL,
    password varchar(255) NOT NULL,
    CONSTRAINT email_unique UNIQUE (email)
);

CREATE TABLE currency (
    id   serial       PRIMARY KEY,
    code varchar(3)   NOT NULL,
    name varchar(255) NOT NULL,
    CONSTRAINT code_unique UNIQUE (code)
);

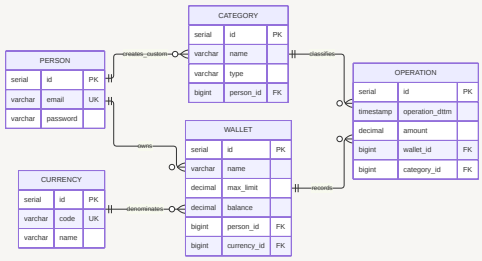
CREATE TABLE wallet (
    id          serial        PRIMARY KEY,
    name        varchar(40)   NOT NULL,
    max_limit   decimal(10,2) NOT NULL,
    balance     decimal(10,2) DEFAULT 0,
    person_id   bigint        REFERENCES person (id) NOT NULL,
    currency_id bigint        REFERENCES currency (id) NOT NULL,
    CONSTRAINT person_id_name_unique UNIQUE (person_id, name)
);

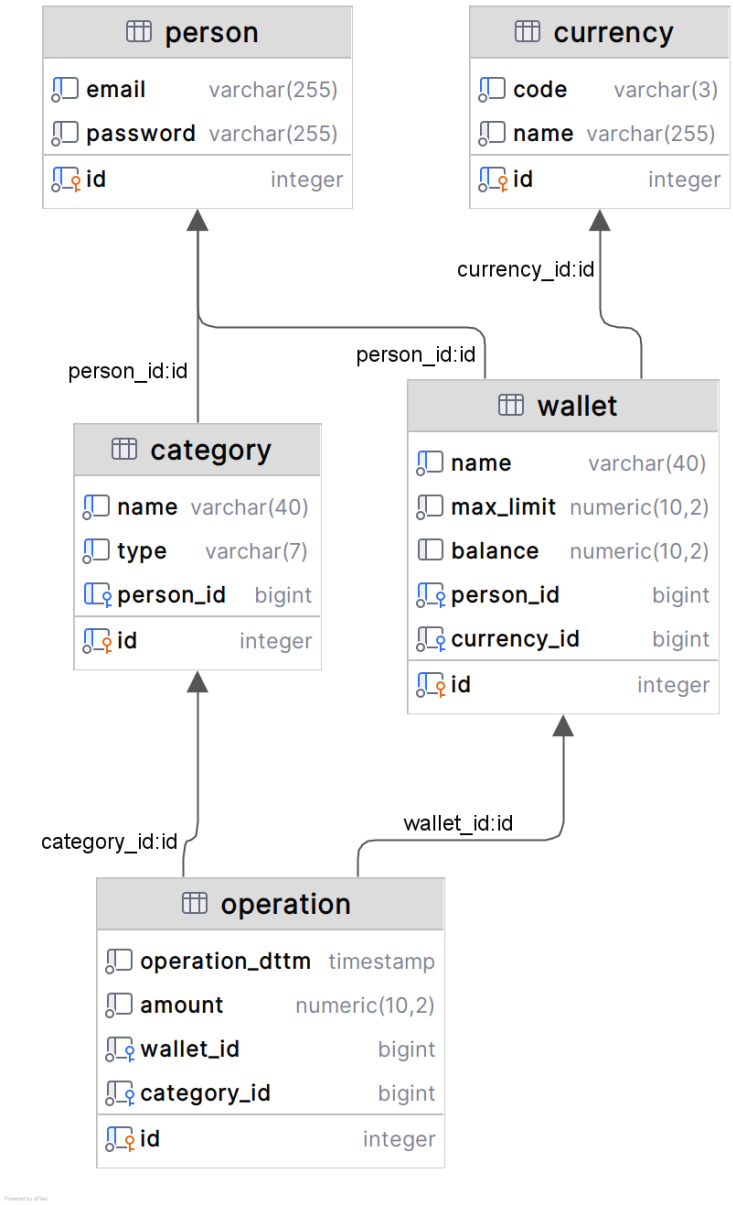
CREATE TABLE category (
    id         serial        PRIMARY KEY,
    name       varchar(40)   NOT NULL,
    type       varchar(7)    NOT NULL,
    person_id  bigint        REFERENCES person (id),
    CONSTRAINT person_id_name_type_unique UNIQUE (person_id, name, type)
);

CREATE TABLE operation (
    id             serial        PRIMARY KEY,
    operation_dttm timestamp     NOT NULL,
    amount         decimal(10,2) NOT NULL,
    wallet_id      bigint        REFERENCES wallet (id) NOT NULL,
```

```
    category_id    bigint        REFERENCES category (id) NOT NULL
);
```

UML





Stored Procedures (PL/pgSQL)

To prevent Time-Of-Check to Time-Of-Use race conditions from the application layer, state-mutating operations are pushed down to the database via Stored Procedures.

The `add_new_operation` procedure performs an atomic evaluation:

1. Locks and retrieves the current `max_limit` and `balance`.
2. Validates that an `EXPENSE` does not exceed the current balance (Overdraft Prevention).
3. Validates that an `INCOME` does not exceed the wallet's `max_limit`.
4. Updates the balance and inserts the immutable `operation` ledger row within a single transaction.

```
CREATE OR REPLACE PROCEDURE add_new_operation(
    operation_dttm timestamp,
    amount numeric(10, 2),
    wallet_id IN wallet.id%TYPE,
    category_id IN category.id%TYPE)
    LANGUAGE plpgsql AS
$$
DECLARE
    max_limit     numeric(10, 2);
    balance       numeric(10, 2);
    category_type varchar(7);
BEGIN
    SELECT w.max_limit, w.balance FROM wallet w WHERE w.id = wallet_id INTO m
ax_limit, balance;
    SELECT c.type FROM category c WHERE c.id = category_id INTO category_typ
e;

    IF (category_type LIKE 'EXPENSE') THEN
        IF (amount > balance) THEN
            RAISE EXCEPTION 'Expense amount cannot be greater than wallet bal
ance';
        END IF;
        UPDATE wallet w SET balance = w.balance - amount WHERE w.id = wallet_
id;
    ELSE
        IF (category_type LIKE 'INCOME') THEN
            IF (amount + balance > max_limit) THEN
                RAISE EXCEPTION 'Income amount + balance cannot exceed wallet
max_limit';
            END IF;
            UPDATE wallet w SET balance = w.balance + amount WHERE w.id = wal
let_id;
        END IF;
    END IF;

    INSERT INTO operation (operation_dttm, amount, wallet_id, category_id)
    VALUES (operation_dttm, amount, wallet_id, category_id);
END
$$;
```

Time-Series Aggregations

Reporting requires dynamic interval calculations. The following query extracts the trailing 30-day cash flow, utilizing Postgres `INTERVAL` logic to automatically handle varying month lengths (e.g., leap years).

```sql
SELECT income, expense
FROM (SELECT SUM(amount)
      FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ?
        AND c.type LIKE 'INCOME'
        AND operation_dttm BETWEEN NOW() - INTERVAL '1' MONTH AND NOW()) AS i
ncome,
     (SELECT SUM(amount)
      FROM operation
      JOIN category c ON c.id = operation.category_id
      WHERE wallet_id = ?
        AND c.type LIKE 'EXPENSE'
        AND operation_dttm BETWEEN NOW() - INTERVAL '1' MONTH AND NOW()) AS e
xpense;
```